

MVC Project: Multilayer Perceptron

Khubaib Ahmed and Roll No: 25i-0016

FAST-NUCES, Islamabad

Abstract. This report details the mathematical derivation, theory, and Python implementation of a Multilayer Perceptron neural network trained on the MNIST dataset. It explores forward propagation, backpropagation, gradient descent variants, and advanced optimizers.

1 Task 1-3: Forward Pass and Backpropagation

The initial phases of this project involved the manual calculation of the forward pass and backpropagation for a multilayer perceptron with two hidden layers. The symbolic equations for the weighted sums, sigmoid activations, and the 12 parameter gradients were successfully derived and computed manually for the initial dataset of 3 samples.

2 Task 4-5: Gradient Descent Variants

2.1 Task 4: Weight Update Theory

During the batch gradient descent updates, the Mean Squared Error (MSE) loss decreased consistently without fluctuating, which suggests that the chosen learning rate ($\eta = 0.1$) is stable and appropriate. However, after only 5 iterations, the network is not fully trained. A better stopping criterion would be to wait for convergence, where the loss stops decreasing significantly between iterations. Furthermore, while Batch Gradient Descent provides a smooth loss curve, its primary disadvantage on real-world datasets with millions of samples is that it is incredibly slow and memory-intensive, requiring a full pass through all data to make a single update.

2.2 Task 5: Mini-Batch Theory

Comparing the loss curves, the mini-batch curve is visibly noisier than the batch curve, but it converges much faster per epoch because it performs multiple updates per pass. Shuffling the dataset at the start of each epoch is critical; without it, the network could memorize the sample order and get stuck in cyclical update patterns. For a dataset of 60,000 samples and a batch size of 32, mini-batch gradient descent performs 1875 updates per epoch. Finally, if the loss surface has a flat region or a saddle point, plain gradient descent will stall completely because the gradient becomes zero and the weights stop updating.

3 Task 6: Optimizers

Upon simulating 10 iterations across different optimizers, Nesterov Accelerated Gradient (NAG) reached the lowest final loss. It significantly outperformed Plain GD, ending with a loss of 0.1950 compared to 0.2970. NAG also converged faster than standard Momentum. While Momentum calculates the gradient at the current position, NAG calculates it at the projected “lookahead” position, allowing it to anticipate flattening slopes and brake early to reduce oscillation.

Alternatively, the Adam optimizer would be preferred over Momentum or NAG when dealing with a highly irregular loss surface or sparse data, as it adapts the learning rate individually for each parameter based on its gradient history.

Finally, scaling this network to the MNIST dataset presents three practical challenges:

1. Massive computational and memory bottlenecks due to the 784 input features, requiring GPU acceleration and mini-batching.
2. The Vanishing Gradient problem, where using the Sigmoid activation in a larger network causes gradients to shrink to near-zero in early layers.
3. A severe risk of overfitting, where the large number of weights causes the network to memorize the training images rather than generalizing.

4 Results (Task 7)

The custom neural network was scaled to 784 inputs and trained on the MNIST dataset for 20 epochs using Mini-Batch Gradient Descent (Batch size: 32, Learning Rate: 0.1). The implementation was completely vectorized using NumPy. The model successfully converged, dropping from an initial MSE loss of 0.2957 to a final MSE loss of 0.0689. Evaluation on the 10,000 sample test set yielded a final classification accuracy of 95.54%.

A Appendix A: Optimizer Simulation Working (Task 6C)

To compute the 10 iterations for the three optimizers across all 12 parameters using Mini-Batch Gradient Descent (B=2), a Python simulation script was utilized to track velocities, lookahead gradients, and parameter updates. The core update logic used to generate the Task 6C iteration table is provided below:

```
import numpy as np

# Simulation parameters
epochs = 10
learning_rate = 0.1
beta = 0.9
```

```
def plain_gd_update(w, grad, lr):  
    return w - lr * grad  
  
def momentum_update(w, v, grad, lr, beta):  
    v_new = beta * v + (1 - beta) * grad  
    w_new = w - lr * v_new  
    return w_new, v_new  
  
def nag_update(w, v, grad_func, lr, beta):  
    # Step 1: Lookahead position  
    w_lookahead = w - lr * beta * v  
  
    # Step 2: Gradient at lookahead  
    g_lookahead = grad_func(w_lookahead)  
  
    # Step 3: Velocity and Weight update  
    v_new = beta * v + (1 - beta) * g_lookahead  
    w_new = w - lr * v_new  
    return w_new, v_new
```